In [30]:
```
import pandas as pd
cust = pd.read_excel("D:\\Data Engineering\\Python\\Custumer_Data.xlsx")
```

In [25]:
```
cust.head()
```

Out[25]:

	ID	Name	Age	Address	Wallet Bal
0	1	Ramesh	32	Ahmedabad	200
1	2	Khilan	23	Delhi	7500
2	3	kaushik	25	Kota	12000
3	4	Chaitali	22	Mumbai	6500
4	5	Hardik	27	Bhopal	600

In [31]:
```
order = pd.read_excel("D:\\Data Engineering\\Python\\Orders_Data.xlsx")
```

In [32]:
```
order.head()
```

Out[32]:

	OID	Date	ID	Amount
0	101	2009-11-20	2	1560
1	102	2009-10-08	3	3000
2	100	2009-10-08	3	1500
3	103	2008-05-20	4	2060

In [5]:
```
cust[["Name","Address"]]
```

Out[5]:

	Name	Address
0	Ramesh	Ahmedabad
1	Khilan	Delhi
2	kaushik	Kota
3	Chaitali	Mumbai
4	Hardik	Bhopal
5	Komal	MP
6	Muffy	Indore

In [6]:
```
order[["Date","Amount"]]
```

Out[6]:

	Date	Amount
0	2009-10-08	3000
1	2009-10-08	1500
2	2009-11-20	1560
3	2008-05-20	2060
4	2008-05-20	1253

In [17]:
```
#Left join with all columns based on common column having same name
```

In [8]:
```
cust_orders_left = pd.merge(cust,order, on = "ID", how = "left" )
cust_orders_left
```

Out[8]:

	ID	Name	Age	Address	Wallet Bal	OID	Date	Amount
0	1	Ramesh	32	Ahmedabad	200	NaN	NaT	NaN
1	2	Khilan	23	Delhi	7500	101.0	2009-11-20	1560.0
2	3	kaushik	25	Kota	12000	102.0	2009-10-08	3000.0
3	3	kaushik	25	Kota	12000	100.0	2009-10-08	1500.0
4	4	Chaitali	22	Mumbai	6500	103.0	2008-05-20	2060.0
5	5	Hardik	27	Bhopal	600	NaN	NaT	NaN
6	6	Komal	26	MP	750	NaN	NaT	NaN
7	7	Muffy	27	Indore	500	NaN	NaT	NaN

In [ ]:
```
#right join with all columns based on common column having same name
```

In [21]:
```
cust_orders_right = pd.merge(cust,order, on = "ID", how = "right")
cust_orders_right
```

Out[21]:

	ID	Name	Age	Address	Wallet Bal	OID	Date	Amount
0	3.0	kaushik	25.0	Kota	12000.0	102	2009-10-08	3000
1	3.0	kaushik	25.0	Kota	12000.0	100	2009-10-08	1500
2	2.0	Khilan	23.0	Delhi	7500.0	101	2009-11-20	1560
3	4.0	Chaitali	22.0	Mumbai	6500.0	103	2008-05-20	2060
4	NaN	NaN	NaN	NaN	NaN	199	2008-05-20	1253

In [ ]:
```
#inner join with all columns based on common column having same name
```

In [22]:
```
cust_orders_inner = pd.merge(cust,order,on = "ID", how = "inner")
cust_orders_inner
```

Out[22]:

	ID	Name	Age	Address	Wallet Bal	OID	Date	Amount
0	2	Khilan	23	Delhi	7500	101	2009-11-20	1560
1	3	kaushik	25	Kota	12000	102	2009-10-08	3000
2	3	kaushik	25	Kota	12000	100	2009-10-08	1500
3	4	Chaitali	22	Mumbai	6500	103	2008-05-20	2060

In [25]:
```
#Left join with selected columns based on common column having same name
```

In [27]:
```
cust_orders_left1 = pd.merge(cust, order, on = "ID", how = "left")[["ID","Name","OID","Amount"]]
cust_orders_left1
```

Out[27]:

	ID	Name	OID	Amount
0	1	Ramesh	NaN	NaN
1	2	Khilan	101.0	1560.0
2	3	kaushik	102.0	3000.0
3	3	kaushik	100.0	1500.0
4	4	Chaitali	103.0	2060.0
5	5	Hardik	NaN	NaN
6	6	Komal	NaN	NaN
7	7	Muffy	NaN	NaN

In [ ]:
```
#right join with selected columns based on common column having same name
```

In [9]:
```
cust_orders_right1 = pd.merge(cust, order, on = "ID", how = "right")[["ID","Name","OID","Amount"]]
cust_orders_right1
```

Out[9]:

	ID	Name	OID	Amount
0	3.0	kaushik	102	3000
1	3.0	kaushik	100	1500
2	2.0	Khilan	101	1560
3	4.0	Chaitali	103	2060
4	NaN	NaN	199	1253

In [ ]:
```
#inner join with selected columns based on common column having same name
```

In [30]:
```
cust_orders_inner1 = pd.merge(cust, order, on = "ID", how = "inner")[["ID","Name","OID","Amount"]]
cust_orders_inner1
```

Out[30]:

	ID	Name	OID	Amount
0	2	Khilan	101	1560
1	3	kaushik	102	3000
2	3	kaushik	100	1500
3	4	Chaitali	103	2060

In [10]:
```
orders = pd.read_excel("D:\\Data Engineering\\Python\\Orders_Data _CID.xlsx")
```

In [32]:
```
orders
```

Out[32]:

	OID	Date	C_ID	Amount
0	102	2009-10-08	3.0	3000
1	100	2009-10-08	3.0	1500
2	101	2009-11-20	2.0	1560
3	103	2008-05-20	4.0	2060
4	199	2008-05-20	NaN	1253

In [ ]:
```
##left join with all columns based on common column having different name
```

In [34]:
```
cust_orders_left_diff = pd.merge(cust, orders, left_on = "ID", right_on = "C_ID", how = "left")
cust_orders_left_diff
```

Out[34]:

	ID	Name	Age	Address	Wallet Bal	OID	Date	C_ID	Amount
0	1	Ramesh	32	Ahmedabad	200	NaN	NaT	NaN	NaN
1	2	Khilan	23	Delhi	7500	101.0	2009-11-20	2.0	1560.0
2	3	kaushik	25	Kota	12000	102.0	2009-10-08	3.0	3000.0
3	3	kaushik	25	Kota	12000	100.0	2009-10-08	3.0	1500.0
4	4	Chaitali	22	Mumbai	6500	103.0	2008-05-20	4.0	2060.0
5	5	Hardik	27	Bhopal	600	NaN	NaT	NaN	NaN
6	6	Komal	26	MP	750	NaN	NaT	NaN	NaN
7	7	Muffy	27	Indore	500	NaN	NaT	NaN	NaN

In [ ]:
```
#right join with all columns based on common column having different name
```

In [11]:
```
cust_orders_right_diff = pd.merge(cust, orders, left_on = "ID", right_on = "C_ID", how = "right")
cust_orders_right_diff
```

Out[11]:

	ID	Name	Age	Address	Wallet Bal	OID	Date	C_ID	Amount
0	3.0	kaushik	25.0	Kota	12000.0	102	2009-10-08	3.0	3000
1	3.0	kaushik	25.0	Kota	12000.0	100	2009-10-08	3.0	1500
2	2.0	Khilan	23.0	Delhi	7500.0	101	2009-11-20	2.0	1560
3	4.0	Chaitali	22.0	Mumbai	6500.0	103	2008-05-20	4.0	2060
4	NaN	NaN	NaN	NaN	NaN	199	2008-05-20	NaN	1253

In [ ]:
```
#inner join with all columns based on common column having different name
```

In [36]:
```
cust_orders_inner_diff = pd.merge(cust, orders, left_on = "ID", right_on = "C_ID", how = "inner")
cust_orders_inner_diff
```

Out[36]:

	ID	Name	Age	Address	Wallet Bal	OID	Date	C_ID	Amount
0	2	Khilan	23	Delhi	7500	101	2009-11-20	2.0	1560
1	3	kaushik	25	Kota	12000	102	2009-10-08	3.0	3000
2	3	kaushik	25	Kota	12000	100	2009-10-08	3.0	1500
3	4	Chaitali	22	Mumbai	6500	103	2008-05-20	4.0	2060

In [ ]:
```
#left join with selected columns based on common column having different name
```

In [12]:
```
cust_orders_left_diff1 = pd.merge(cust, orders, left_on = "ID", right_on = "C_ID",how = "left")[["ID","Name","OID","Amount"]
cust_orders_left_diff1
```

Out[12]:

	ID	Name	OID	Amount
0	1	Ramesh	NaN	NaN
1	2	Khilan	101.0	1560.0
2	3	kaushik	102.0	3000.0
3	3	kaushik	100.0	1500.0
4	4	Chaitali	103.0	2060.0
5	5	Hardik	NaN	NaN
6	6	Komal	NaN	NaN
7	7	Muffy	NaN	NaN

In [ ]:
```
#right join with selected columns based on common column having different name
```

In [13]:
```
cust_orders_right_diff1 = pd.merge(cust, orders, left_on = "ID", right_on = "C_ID", how = "right")[["ID","Name","OID","Date"
cust_orders_right_diff1
```

Out[13]:

	ID	Name	OID	Date	Amount
0	3.0	kaushik	102	2009-10-08	3000
1	3.0	kaushik	100	2009-10-08	1500
2	2.0	Khilan	101	2009-11-20	1560
3	4.0	Chaitali	103	2008-05-20	2060
4	NaN	NaN	199	2008-05-20	1253

In [ ]:
```
#inner join with selected columns based on common column having different name
```

In [14]:
```
cust_orders_inner_diff1 = pd.merge(cust, orders, left_on = "ID", right_on = "C_ID", how = "inner")[["ID","Name","OID","Date"
cust_orders_inner_diff1
```

Out[14]:

	ID	Name	OID	Date	Amount
0	2	Khilan	101	2009-11-20	1560
1	3	kaushik	102	2009-10-08	3000
2	3	kaushik	100	2009-10-08	1500
3	4	Chaitali	103	2008-05-20	2060

In [ ]:
```
#inner join with selected columns based on common column having different name with groupby condition
```

In [15]:
```
cust_orders_inner_diff1 = pd.merge(cust, orders, left_on = "ID", right_on = "C_ID", how = "inner")[["ID","Name","OID","Amoun
                          .query('ID == 3')
cust_orders_inner_diff1
```

Out[15]:

	ID	Name	OID	Amount
1	3	kaushik	102	3000
2	3	kaushik	100	1500

In [ ]:
```
#Join with all columns based on common column having different name  will same common columns names
```

In [16]:
```
ordercust = pd.read_excel("D:\\Data Engineering\\Python\\Orders_Data _CID_Name.xlsx")
```

In [47]:
```
ordercust
```

Out[47]:

	OID	Date	Name	C_ID	Amount
0	102	2009-10-08	kaushik	3.0	3000
1	100	2009-10-08	kaushik	3.0	1500
2	101	2009-11-20	Khilan	2.0	1560
3	103	2008-05-20	Chaitali	4.0	2060
4	199	2008-05-20	NaN	NaN	1253

In [51]:
```
cust_orde_suffix = pd.merge(cust, ordercust, left_on = "ID", right_on = "C_ID",suffixes = ('_cust','_order'), how = "inner")
cust_orde_suffix
```

Out[51]:

	ID	Name_cust	Age	Address	Wallet Bal	OID	Date	Name_order	C_ID	Amount
0	2	Khilan	23	Delhi	7500	101	2009-11-20	Khilan	2.0	1560
1	3	kaushik	25	Kota	12000	102	2009-10-08	kaushik	3.0	3000
2	3	kaushik	25	Kota	12000	100	2009-10-08	kaushik	3.0	1500
3	4	Chaitali	22	Mumbai	6500	103	2008-05-20	Chaitali	4.0	2060

In [ ]:
```
#if i want to rename common column without underscore/suffixes
```

In [61]:
```
cust_order_suffix2 = pd.merge(cust, ordercust, left_on = "ID", right_on = "C_ID",suffixes=('_cust','_order') ,how = "inner")\
                      .rename(columns = {'Name_cust': "Cust_Name", 'Name_order': "Ordered_Customer"})
cust_order_suffix2
```

Out[61]:

	ID	Cust_Name	Age	Address	Wallet Bal	OID	Date	Ordered_Customer	C_ID	Amount
0	2	Khilan	23	Delhi	7500	101	2009-11-20	Khilan	2.0	1560
1	3	kaushik	25	Kota	12000	102	2009-10-08	kaushik	3.0	3000
2	3	kaushik	25	Kota	12000	100	2009-10-08	kaushik	3.0	1500
3	4	Chaitali	22	Mumbai	6500	103	2008-05-20	Chaitali	4.0	2060

In [ ]:
```
#1) the joined tablemshould sort with age column
#2) merge vs mergeasof vs merge_ordered get the diff b/w these three
```

In [68]:
```
cust_order_rename_agesort_asc = pd.merge(cust, ordercust, left_on = "ID", right_on = "C_ID", how = "inner")\
                                  .rename(columns = {"Name_x":"Cust_Name", "Name_y":"Ordered_customer"})\
                                  .sort_values("Age",ascending=True)
cust_order_rename_agesort
```

Out[68]:

	ID	Cust_Name	Age	Address	Wallet Bal	OID	Date	Ordered_customer	C_ID	Amount
3	4	Chaitali	22	Mumbai	6500	103	2008-05-20	Chaitali	4.0	2060
0	2	Khilan	23	Delhi	7500	101	2009-11-20	Khilan	2.0	1560
1	3	kaushik	25	Kota	12000	102	2009-10-08	kaushik	3.0	3000
2	3	kaushik	25	Kota	12000	100	2009-10-08	kaushik	3.0	1500

In [69]:
```
cust_order_rename_agesort_desc = pd.merge(cust, ordercust, left_on = "ID", right_on = "C_ID", how = "inner")\
                                   .rename(columns = {'Name_x':"Cust_Name",'Name_y':"Ordered_customer"})\
                                   .sort_values("Age",ascending=False)
cust_order_rename_agesort_desc
```

Out[69]:

	ID	Cust_Name	Age	Address	Wallet Bal	OID	Date	Ordered_customer	C_ID	Amount
1	3	kaushik	25	Kota	12000	102	2009-10-08	kaushik	3.0	3000
2	3	kaushik	25	Kota	12000	100	2009-10-08	kaushik	3.0	1500
0	2	Khilan	23	Delhi	7500	101	2009-11-20	Khilan	2.0	1560
3	4	Chaitali	22	Mumbai	6500	103	2008-05-20	Chaitali	4.0	2060

In [ ]:
```
#merge vs mergeasof vs merge_ordered get the diff b/w these three
```

In [18]:
```
cust.head()
```

Out[18]:

	ID	Name	Age	Address	Wallet Bal
0	1	Ramesh	32	Ahmedabad	200
1	2	Khilan	23	Delhi	7500
2	3	kaushik	25	Kota	12000
3	4	Chaitali	22	Mumbai	6500
4	5	Hardik	27	Bhopal	600

In [48]:
```
order.head()
```

Out[48]:

	OID	Date	ID	Amount
0	101	2009-11-20	2	1560
1	102	2009-10-08	3	3000
2	100	2009-10-08	3	1500
3	103	2008-05-20	4	2060

In [49]:
```
pd.merge_asof(cust, order, on = "ID") #full outer join
```

Out[49]:

	ID	Name	Age	Address	Wallet Bal	OID	Date	Amount
0	1	Ramesh	32	Ahmedabad	200	NaN	NaT	NaN
1	2	Khilan	23	Delhi	7500	101.0	2009-11-20	1560.0
2	3	kaushik	25	Kota	12000	100.0	2009-10-08	1500.0
3	4	Chaitali	22	Mumbai	6500	103.0	2008-05-20	2060.0
4	5	Hardik	27	Bhopal	600	103.0	2008-05-20	2060.0
5	6	Komal	26	MP	750	103.0	2008-05-20	2060.0
6	7	Muffy	27	Indore	500	103.0	2008-05-20	2060.0

In [37]:
```
pd.merge_asof(cust, order, on = "ID",allow_exact_matches=True) #left and right key must in same order (soring should be corre
```

Out[37]:

	ID	Name	Age	Address	Wallet Bal	OID	Date	Amount
0	1	Ramesh	32	Ahmedabad	200	NaN	NaT	NaN
1	2	Khilan	23	Delhi	7500	101.0	2009-11-20	1560.0
2	3	kaushik	25	Kota	12000	100.0	2009-10-08	1500.0
3	4	Chaitali	22	Mumbai	6500	103.0	2008-05-20	2060.0
4	5	Hardik	27	Bhopal	600	103.0	2008-05-20	2060.0
5	6	Komal	26	MP	750	103.0	2008-05-20	2060.0
6	7	Muffy	27	Indore	500	103.0	2008-05-20	2060.0

In [40]:
```
pd.merge_asof(cust, order, on = "ID",allow_exact_matches=False)
```

Out[40]:

	ID	Name	Age	Address	Wallet Bal	OID	Date	Amount
0	1	Ramesh	32	Ahmedabad	200	NaN	NaT	NaN
1	2	Khilan	23	Delhi	7500	NaN	NaT	NaN
2	3	kaushik	25	Kota	12000	101.0	2009-11-20	1560.0
3	4	Chaitali	22	Mumbai	6500	100.0	2009-10-08	1500.0
4	5	Hardik	27	Bhopal	600	103.0	2008-05-20	2060.0
5	6	Komal	26	MP	750	103.0	2008-05-20	2060.0
6	7	Muffy	27	Indore	500	103.0	2008-05-20	2060.0

In [41]:
```
pd.merge_asof(cust, order, on = "ID",how="right",allow_exact_matches=False) # not accept how..by default it is taking fulljo
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[41], line 1
----> 1 pd.merge_asof(cust, order, on = "ID",how="right",allow_exact_matches=False)

TypeError: merge_asof() got an unexpected keyword argument 'how'
```

In [42]:
```
pd.merge_asof(cust, order, on = "ID",direction="nearest")
```

Out[42]:

	ID	Name	Age	Address	Wallet Bal	OID	Date	Amount
0	1	Ramesh	32	Ahmedabad	200	101	2009-11-20	1560
1	2	Khilan	23	Delhi	7500	101	2009-11-20	1560
2	3	kaushik	25	Kota	12000	100	2009-10-08	1500
3	4	Chaitali	22	Mumbai	6500	103	2008-05-20	2060
4	5	Hardik	27	Bhopal	600	103	2008-05-20	2060
5	6	Komal	26	MP	750	103	2008-05-20	2060
6	7	Muffy	27	Indore	500	103	2008-05-20	2060

In [ ]:
```
#merge_ordered
```

In [43]:
```
pd.merge_ordered(cust, order, on = "ID",fill_method="ffill") #filling
```

Out[43]:

	ID	Name	Age	Address	Wallet Bal	OID	Date	Amount
0	1	Ramesh	32	Ahmedabad	200	NaN	NaT	NaN
1	2	Khilan	23	Delhi	7500	101.0	2009-11-20	1560.0
2	3	kaushik	25	Kota	12000	102.0	2009-10-08	3000.0
3	3	kaushik	25	Kota	12000	100.0	2009-10-08	1500.0
4	4	Chaitali	22	Mumbai	6500	103.0	2008-05-20	2060.0
5	5	Hardik	27	Bhopal	600	103.0	2008-05-20	2060.0
6	6	Komal	26	MP	750	103.0	2008-05-20	2060.0
7	7	Muffy	27	Indore	500	103.0	2008-05-20	2060.0

In [50]:
```
pd.merge_ordered(cust, order, on = "ID",fill_method="ffill",how="outer") # we can use how..condition # by deafult outerjoin
```

Out[50]:

	ID	Name	Age	Address	Wallet Bal	OID	Date	Amount
0	1	Ramesh	32	Ahmedabad	200	NaN	NaT	NaN
1	2	Khilan	23	Delhi	7500	101.0	2009-11-20	1560.0
2	3	kaushik	25	Kota	12000	102.0	2009-10-08	3000.0
3	3	kaushik	25	Kota	12000	100.0	2009-10-08	1500.0
4	4	Chaitali	22	Mumbai	6500	103.0	2008-05-20	2060.0
5	5	Hardik	27	Bhopal	600	103.0	2008-05-20	2060.0
6	6	Komal	26	MP	750	103.0	2008-05-20	2060.0
7	7	Muffy	27	Indore	500	103.0	2008-05-20	2060.0